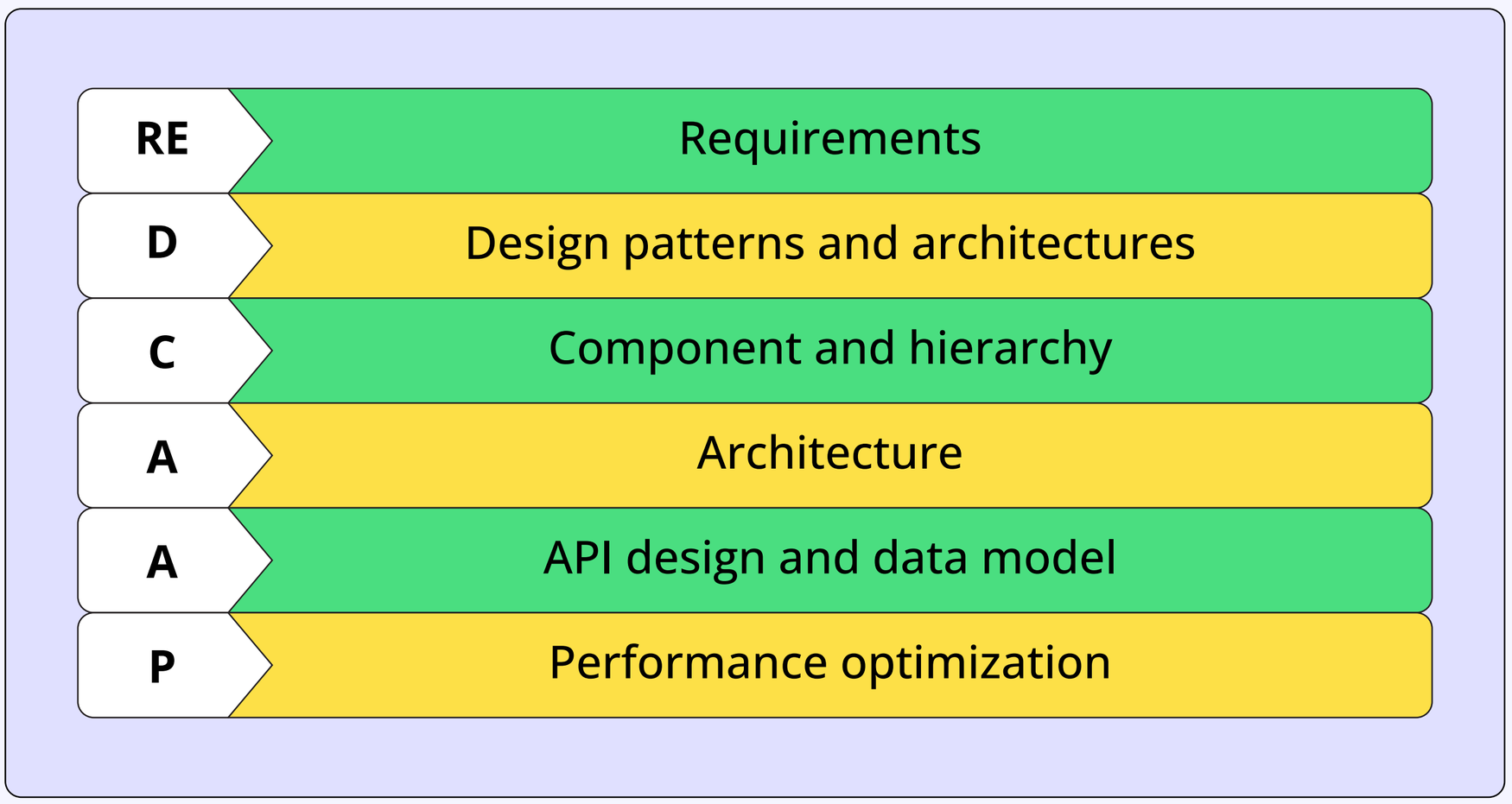
Frontend System Design Interview Framework

Building scalable frontend systems is complex, but the REDCAAP method provides a clear, structured approach to break down design challenges and ensure consistent, maintainable solutions.

The REDCAAP Approach for Designing Frontend Systems

- **REDCAAP:** A structured framework for solving frontend System Design problems.
- **Purpose:** Break down complex systems into manageable, scalable components.



The breakdown of the REDCAAP framework

Understanding the REDCAAP

1. Requirements (RE)

- **Functional requirments:** Define the core features and interactions.
  - **Example:** In a chat application, these can include typing indicators, message read receipts, and user presence updates.
- **Nonfunctional requirments:** Focus on system performance, scalability, and responsiveness.
  - **Example:** For a video streaming application, these include low latency, high availability, and adaptive video quality.



2. Design Patterns and Atrchitectures (D)

In this step, we choose suitable design patterns and architectures that help organize the frontend system.

i) **MVC vs. MVVM vs. MVP:** These architectural patterns define how UI, logic, and data interact. Prioritize the separation of concerns, ease of testing, and handling dynamic updates.

ii) **SPA vs. MPA:** SPAs load content dynamically on one page for smoother interactions. MPAs load full pages separately, offering better SEO and traditional navigation. Choose SPA for dynamic UX, MPA for SEO, and simplicity.

iii) **Monolithic vs. micro frontend:** Pick a monolithic SPA for simplicity or micro-frontends for large teams and modular development.

iv) **Component-based vs. monolithic UI:** Use component-based UI for reusable, modular design; choose monolithic UI for simpler, centralized architecture.

v) **Unidirectional vs. bidirectional data flow:** Use unidirectional flow for predictable state management and debugging; bidirectional flow is better for real-time, interactive applications.

Key takeaway: Select design patterns that are scalable, modular, and future-proof, ensuring maintainability and flexibility as the system grows.

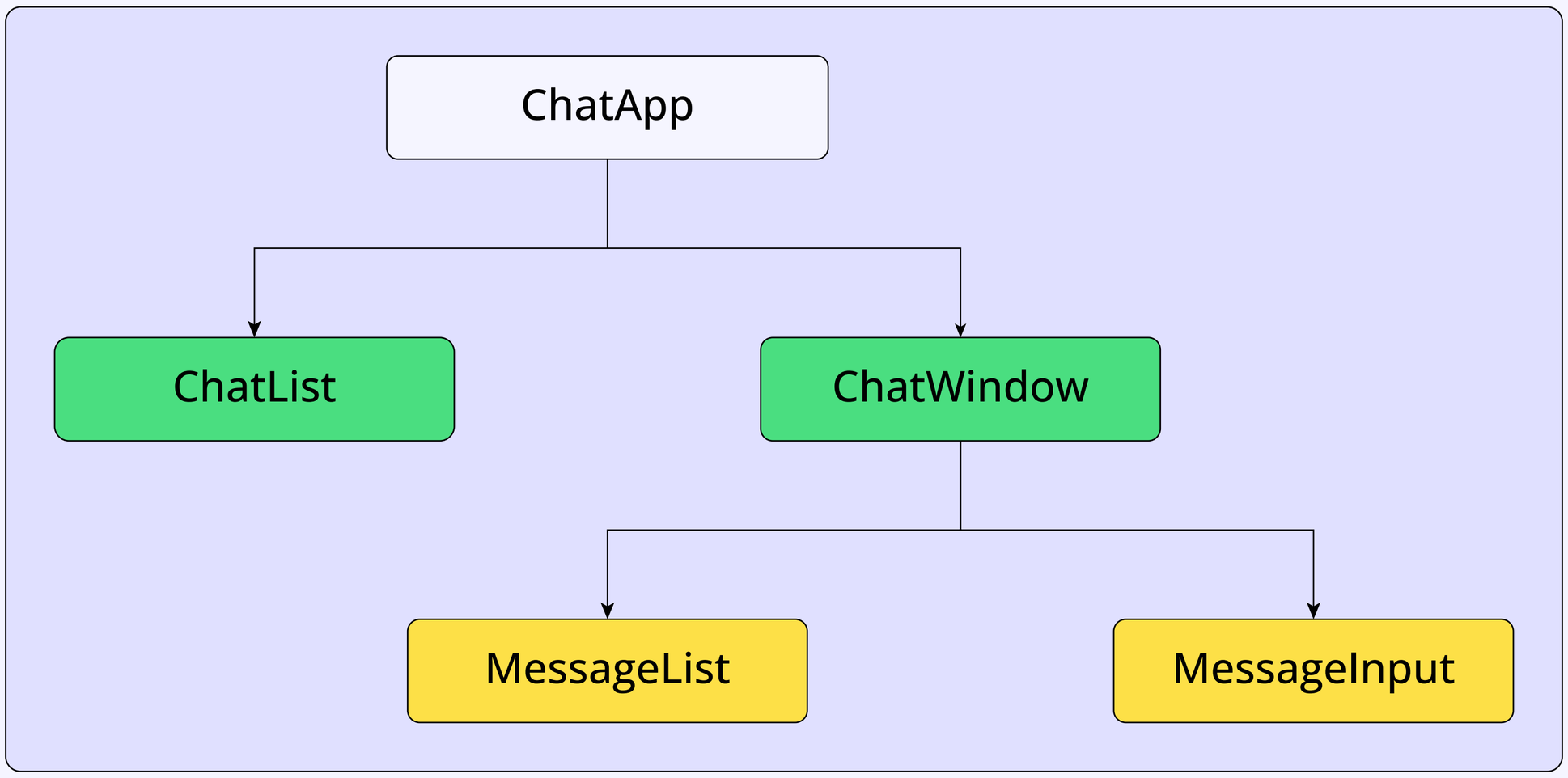
3. Component and Hierarchy (C)

Build the UI as modular, reusable components with clear roles, arranged in a hierarchy to improve maintainability and optimize performance.

i) **Modular components:** Break UI into reusable, independent parts for easier development, testing, and optimization.

ii) **Component hierarchy:** Establish clear parent-child relationships to manage state efficiently and minimize unnecessary updates.

Consider a chat app structured into modular components:

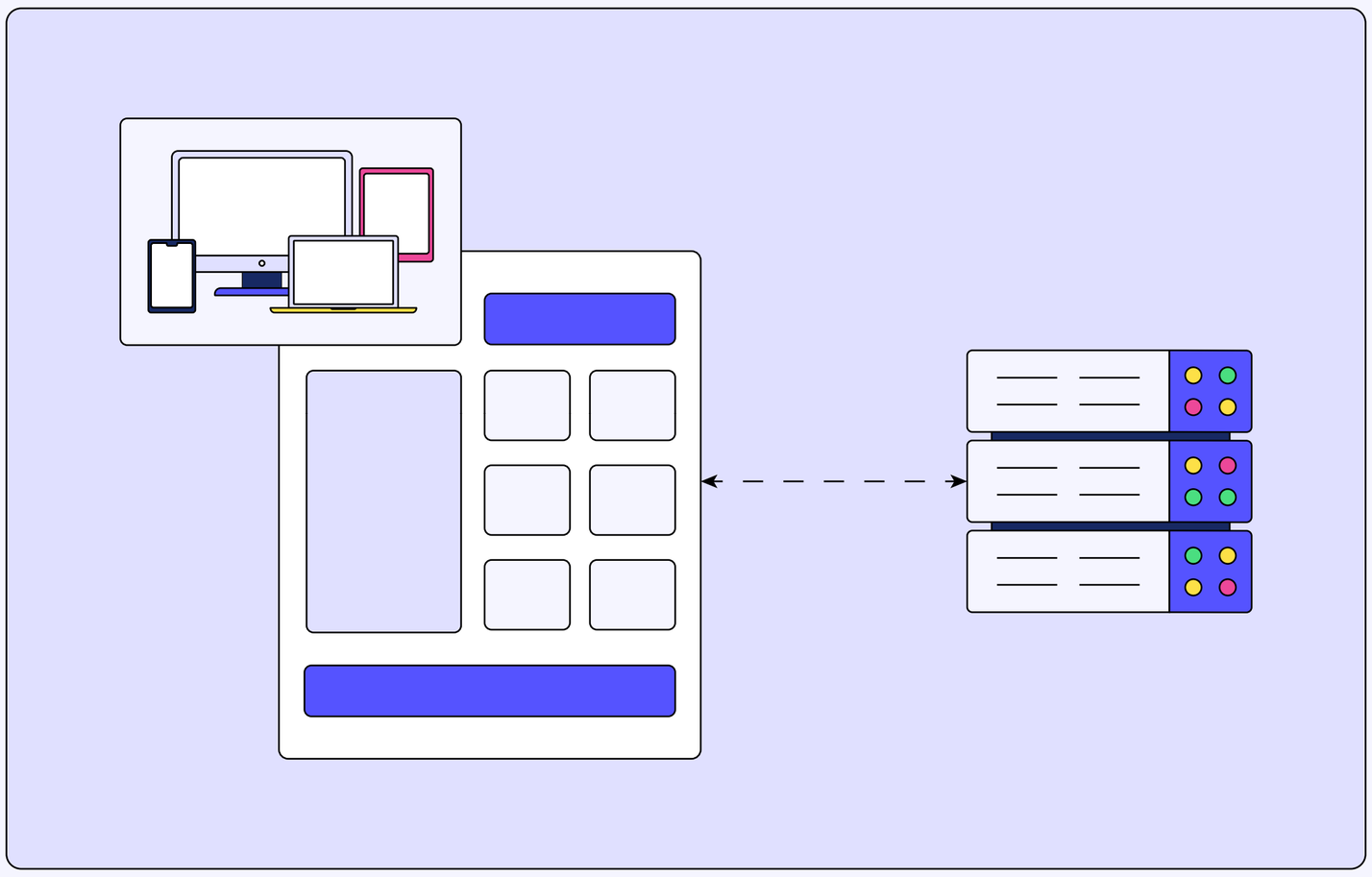


Component’s hierarchy

Key takeaway: A component-based structure enhances maintainability and performance.

4. Architecture (A)

- **Modular frontend to backend mapping:** Ensure that frontend components integrate smoothly with backend services, creating a responsive, maintainable, and scalable system.



Modular frontend mapping to backend (services)

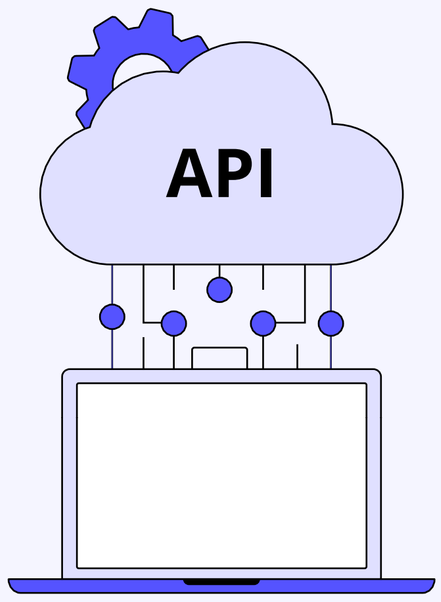
Key takeaway: Achieve seamless and scalable integration between the frontend and backend.

5. API Design (A)

i. **Choose the right API style:** Use RESTful or GraphQL APIs based on the system's needs.

ii. **Modular, purpose-driven API endpoints:** Design APIs to minimize complexity and ensure efficient data flow.

iii. **Optimized data models:** Prevent over-fetching or under-fetching data, ensuring the frontend gets the necessary data in the most efficient format.



Key takeaway: Design efficient, scalable, and maintainable APIs to ensure smooth frontend-backend communication.

6. Performance Optimization (P)

The performance optimization includes the following key areas:

- **Performance:** Ensure fast rendering and optimized network requests.
- **Compatibility:** Support multiple devices, browsers, and screen resolutions.
- **Accessibility:** Design inclusive applications that accommodate all users' needs.
- **Security:** Protect user data and prevent vulnerabilities.
- **Localization:** Support multiple languages, currency formats, and regional settings.

Key takeaway: Optimize for seamless, responsive, and efficient user experiences across all devices.

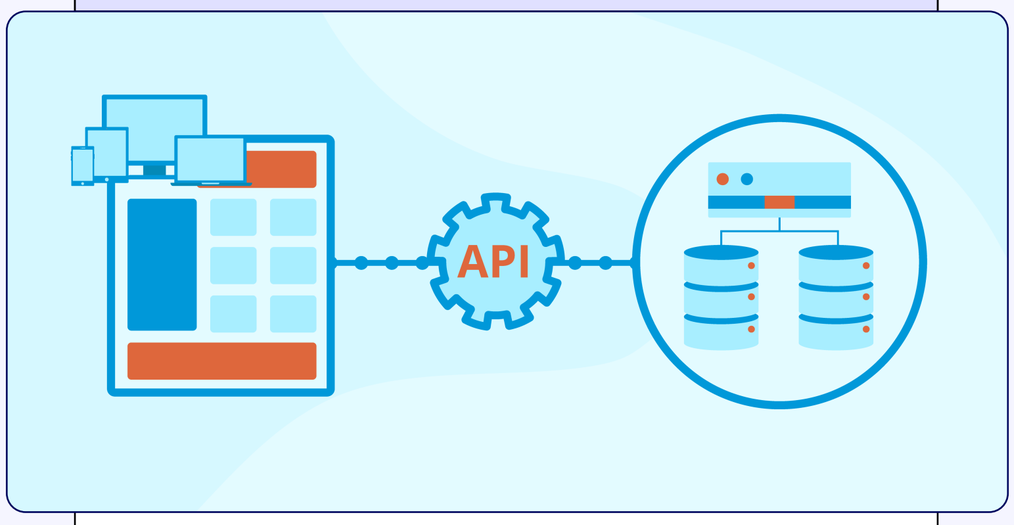
Final Thought

The REDCAAP framework streamlines the design process, ensuring that all critical aspects are thoroughly addressed for a successful frontend system.

System Design Courses

If you want to deepen your knowledge and master frontend System Design, explore our following courses to gain practical insights and hands-on strategies.

Grokking the Frontend System Design Interview



Grokking the Modern System Design Interview

